Strict Mode

Strict mode (see Strict Mode) enables more warnings and makes JavaScript a cleaner language (non-strict mode is sometimes called “sloppy mode”). To switch it on, type the following line first in a JavaScript file or a <script> tag:

```
'use strict';
```

You can also enable strict mode per function:

```
function functionInStrictMode() {
    'use strict';
}
```

Variable Scoping and Closures

In JavaScript, you declare variables via var before using them:

```
> var x;
> x
undefined
> y
ReferenceError: y is not defined
```

You can declare and initialise several variables with single var statement:

```
var x = 1, y = 2, z = 3;
```

Best Practice: using one statement per variable.

```
var x = 1;
var y = 2;
var z = 3;
```

Because of hoisting (see Variables Are Hoisted), it is usually best to declare variables at the beginning of a function.

Variables Are Function-Scoped

The scope of a variable is always the complete function (as opposed to the current block). For example:

```
function foo() {
    var x = -512;
    if (x < 0) {  // (1)
        var tmp = -x;
        ...
    }
    console.log(tmp);  // 512
}
```

We can see that the variable tmp is not restricted to the block starting in line (1); it exists until the end of the function.

Variables Are Hoisted

Each variable declaration is *hoisted*: the declaration is moved to the beginning of the function, but assignments that it makes stay put. As an example, consider the variable declaration in line (1) in the following function:

```
function foo() {
    console.log(tmp); // undefined
    if (false) {
        var tmp = 3;  // (1)
    }
}
```

Internally, the preceding function is executed like this:

```
function foo() {
    var tmp;  // hoisted declaration
    console.log(tmp);
    if (false) {
        tmp = 3;  // assignment stays put
    }
}
```

Closures

Each function stays connected to the variables of the functions that surround it, even after it leaves the scope in which it was created. For example:

```
function createIncrementor(start) {
    return function () {  // (1)
        start++;
        return start;
    }
}
```

The function starting in line (1) leaves the context in which it was created, but stays connected to a live version of start:

```
> var inc = createIncrementor(5);
> inc()
6
> inc()
7
> inc()
8
```

A *closure* is a function plus the connection to the variables of its surrounding scopes. Thus, what createIncrementor() returns is a closure.

JavaScript Form Validation

HTML form validation can be done by JavaScript.

If a form field (first_name) is empty, this function alerts a message, and returns false, to prevent the form from being submitted:

```
<!DOCTYPE html>
<html>
<head>
<script>
function validateForm() {
  var x = document.forms[“myForm"]["first_name"].value;
  if (x == "") {
    alert("Name must be filled out");
    return false;
  }
}
</script>
</head>
<body>

<form name="myForm" action="/action_page.php" onsubmit="return validateForm()" method="post">
  Name: <input type="text" name=“first_name">
  <input type="submit" value="Submit">
</form>

</body>
</html>
```

Data Validation

Data validation is the process of ensuring that user input is clean, correct, and useful.

Typical validation tasks are:

- has the user filled in all required fields?
- has the user entered a valid date?
- has the user entered text in a numeric field?

Most often, the purpose of data validation is to ensure correct user input.

Validation can be defined by many different methods, and deployed in many different ways.

Server side validation is performed by a web server, after input has been sent to the server.

Client side validation is performed by a web browser, before input is sent to a web server.

JavaScript FunctionDefinitions

JavaScript functions are **defined** with the `function` keyword.

You can use a function **declaration** or a function **expression.**

Function Declarations

```
function functionName(parameters) {
  // code to be executed
}
```

Declared functions are not executed immediately. They are "saved for later use", and will be executed later, when they are invoked (called upon).

Semicolons are used to separate executable JavaScript statements.
Since a function declaration is not an executable statement, it is not common to end it with a semicolon.

Function Expressions

A JavaScript function can also be defined using an **expression**.

A function expression can be stored in a variable:

```
var x = function (a, b) {return a * b};
```

After a function expression has been stored in a variable, the variable can be used as a function:

```
var x = function (a, b) {return a * b};
var z = x(4, 3);
```

The function above is actually an **anonymous function** (a function without a name).

Functions stored in variables do not need function names. They are always invoked (called) using the variable name.

The Function() Constructor

As you have seen in the previous examples, JavaScript functions are defined with the `function` keyword.

Functions can also be defined with a built-in JavaScript function constructor called `Function()`.

```
var myFunction = new Function("a", "b", "return a
* b");

var x = myFunction(4, 3);
```

Function Hoisting

Hoisting is JavaScript's default behavior of moving **declarations** to the top of the current scope.

Hoisting applies to variable declarations and to function declarations.Because of this, JavaScript functions can be called before they are declared:

```
myFunction(5);

function myFunction(y) {
  return y * y;
}
```

Functions defined using an expression are not hoisted.

Self-Invoking Functions

Function expressions can be made "self-invoking".

A self-invoking expression is invoked (started) automatically, without being called.

Function expressions will execute automatically if the expression is followed by ().

You cannot self-invoke a function declaration.

You have to add parentheses around the function to indicate that it is a function expression:

```
(function () {
  var x = "Hello!!";  // I will invoke myself
})();
```

The function above is actually an **anonymous self-invoking function** (function without name).

Functions Can Be Used as Values

JavaScript functions can be used as values:

```
function myFunction(a, b) {
  return a * b;
}

var x = myFunction(4, 3);
```

JavaScript functions can be used in expressions:

```
function myFunction(a, b) {
  return a * b;
}

var x = myFunction(4, 3) * 2;
```

Functions are Objects

The `typeof` operator in JavaScript returns "function" for functions.

But, JavaScript functions can best be described as objects.

JavaScript functions have both **properties** and **methods**.

The `arguments.length` property returns the number of arguments received when the function was invoked:

```
function myFunction(a, b) {
  return arguments.length;
}
```

The `toString()` method returns the function as a string:

```
function myFunction(a, b) {
  return a * b;
}

var txt = myFunction.toString();
```

A function defined as the property of an object, is called a method to the object.
A function designed to create new objects, is called an object constructor.

Arrow Functions

Arrow functions allows a short syntax for writing function expressions.

You don't need the `function` keyword, the `return` keyword, and the **curly brackets**.

```
// ES5
var x = function(x, y) {
  return x * y;
}

// ES6
const x = (x, y) => x * y;
```

Arrow functions do not have their own `this`. They are not well suited for defining **object methods**.

Arrow functions are not hoisted. They must be defined **before** they are used.

Using `const` is safer than using `var`, because a function expression is always constant value.

You can only omit the `return` keyword and the curly brackets if the function is a single statement. Because of this, it might be a good habit to always keep them:

```
const x = (x, y) => { return x * y };
```

JavaScript Function Invocation

The code inside a JavaScript `function` will execute when "something" invokes it.

Invoking a JavaScript Function

The code inside a function is not executed when the function is **defined**.

The code inside a function is executed when the function is **invoked**.

It is common to use the term "**call a function**" instead of "**invoke a function**".

It is also common to say "call upon a function", "start a function", or "execute a function".

In this tutorial, we will use **invoke**, because a JavaScript function can be invoked without being called.

```
function myFunction(a, b) {
  return a * b;
}
myFunction(10, 2);              // Will return 20
```

The function above does not belong to any object. But in JavaScript there is always a default global object.

In HTML the default global object is the HTML page itself, so the function above "belongs" to the HTML page.

In a browser the page object is the browser window. The function above automatically becomes a window function.

myFunction() and window.myFunction() is the same function:

```
function myFunction(a, b) {
  return a * b;
}
window.myFunction(10, 2);  // Will also return 20
```

The *this* Keyword

In JavaScript, the thing called `this`, is the object that "owns" the current code.

The value of `this`, when used in a function, is the object that "owns" the function.

Note that `this` is not a variable. It is a keyword. You cannot change the value of `this`.

Tip: Read more about the `this` keyword at JS this Keyword.

The Global Object

When a function is called without an owner object, the value of `this` becomes the global object.

In a web browser the global object is the browser window.

This example returns the window object as the value of `this`:

```
var x = myFunction();//x will be the window object

function myFunction() {
  return this;
}
```

Invoking a function as a global function, causes the value of **this** to be the global object.
Using the window object as a variable can easily crash your program.

Invoking a Function as a Method

In JavaScript you can define functions as object methods.

The following example creates an object (**myObject**), with two properties (**firstName** and **lastName**), and a method (**fullName**):

```
var myObject = {
  firstName:"John",
  lastName: "Doe",
  fullName: function () {
    return this.firstName + " " + this.lastName;
  }
}
myObject.fullName();     // Will return "John Doe"
```

The **fullName** method is a function. The function belongs to the object. **myObject** is the owner of the function.

The thing called `this`, is the object that "owns" the JavaScript code. In this case the value of `this` is **myObject**.

Test it! Change the **fullName** method to return the value of `this`:

```
var myObject = {
  firstName:"John",
  lastName: "Doe",
  fullName: function () {
    return this;
  }
}
myObject.fullName();

// Will return [object Object] (the owner object)
```

Invoking a function as an object method, causes the value of `this` to be the object itself.

Invoking a Function with a Function Constructor

If a function invocation is preceded with the `new` keyword, it is a constructor invocation.

It looks like you create a new function, but since JavaScript functions are objects you actually create a new object:

```
// This is a function constructor:
function myFunction(arg1, arg2) {
  this.firstName = arg1;
  this.lastName  = arg2;
}

// This creates a new object
var x = new myFunction("John", "Doe");
x.firstName;                             // Will
return "John"
```

A constructor invocation creates a new object. The new object inherits the properties and methods from its constructor.

The `this` keyword in the constructor does not have a value.
The value of `this` will be the new object created when the function is invoked.

JavaScript Function Call

Method Reuse

With the `call()` method, you can write a method that can be used on different objects.

All Functions are Methods

In JavaScript all functions are object methods.

If a function is not a method of a JavaScript object, it is a function of the global object (see previous chapter).

The example below creates an object with 3 properties, firstName, lastName, fullName.

```
var person = {
  firstName:"John",
  lastName: "Doe",
  fullName: function () {
    return this.firstName + " " + this.lastName;
  }
}
person.fullName();   // Will return "John Doe"
```

The **this** Keyword

In a function definition, `this` refers to the "owner" of the function.

In the example above, `this` is the **person object** that "owns" the **fullName** function.

In other words, **this.firstName** means
the **firstName** property of **this object**.

Read more about the `this` keyword at JS this Keyword.

The JavaScript call() Method

The `call()` method is a predefined JavaScript method.

It can be used to invoke (call) a method with an owner object as an argument (parameter).

With `call()`, an object can use a method belonging to another object.

This example calls the **fullName** method of person, using it on **person1**:

```
var person = {
  fullName: function() {
    return this.firstName + " " + this.lastName;
  }
}
var person1 = {
  firstName:"John",
  lastName: "Doe",
}
var person2 = {
  firstName:"Mary",
  lastName: "Doe",
}
person.fullName.call(person1);
// Will return "John Doe”
```

This example calls the **fullName** method of person, using it on **person2**:

```
var person = {
  fullName: function() {
    return this.firstName + " " + this.lastName;
  }
}
var person1 = {
  firstName:"John",
  lastName: "Doe",
}
var person2 = {
  firstName:"Mary",
  lastName: "Doe",
}
person.fullName.call(person2);
// Will return "Mary Doe”
```

The call() Method with Arguments

The `call()` method can accept arguments:

```
var person = {
  fullName: function(city, country) {
    return this.firstName + "
" + this.lastName + "," + city + "," + country;
  }
}
var person1 = {
  firstName:"John",
  lastName: "Doe",
}
person.fullName.call(person1, "Oslo", "Norway");
```

JavaScript Function Apply

Method Reuse

With the `apply()` method, you can write a method that can be used on different objects.

The JavaScript apply() Method

The `apply()` method is similar to the `call()` method (previous chapter).

In this example the **fullName** method of **person** is **applied** on **person1**:

```
var person = {
  fullName: function() {
    return this.firstName + " " + this.lastName;
  }
}
var person1 = {
  firstName: "Mary",
  lastName: "Doe",
}
person.fullName.apply(person1);
// Will return "Mary Doe"
```

The Difference Between call() and apply()

The difference is:

The `call()` method takes arguments **separately**.

The `apply()` method takes arguments as an **array**.

The apply() method is very handy if you want to use an array instead of an argument list.

The apply() Method with Arguments

The `apply()` method accepts arguments in an array:

```
var person = {
  fullName: function(city, country) {
    return this.firstName + "
" + this.lastName + "," + city + "," + country;
  }
}
var person1 = {
  firstName:"John",
  lastName: "Doe",
}
person.fullName.apply(person1,
["Oslo", "Norway"]);
```

Compared with the `call()` method:

```
var person = {
  fullName: function(city, country) {
    return this.firstName + "
" + this.lastName + "," + city + "," + country;
  }
}
var person1 = {
  firstName:"John",
  lastName: "Doe",
}
person.fullName.call(person1, "Oslo", "Norway");
```

Simulate a Max Method on Arrays

You can find the largest number (in a list of numbers) using the `Math.max()` method:

```
Math.max(1,2,3);   // Will return 3
```

Since JavaScript **arrays** do not have a max() method, you can apply the `Math.max()` method instead.

```
Math.max.apply(null, [1,2,3]);
// Will also return 3
Math.max.apply(Math, [1,2,3]);
// Will also return 3
Math.max.apply(" ", [1,2,3]);
// Will also return 3
Math.max.apply(0, [1,2,3]);
// Will also return 3
```

JavaScript Strict Mode

In JavaScript strict mode, if the first argument of the `apply()` method is not an object, it becomes the owner (object) of the invoked function. In "non-strict" mode, it becomes the global object.

JavaScript Closures

JavaScript variables can belong to the **local** or **global** scope.

Global variables can be made local (private) with **closures**.

Global Variables

A `function` can access all variables defined **inside** the function, like this:

```
function myFunction() {
  var a = 4;
  return a * a;
}
```

But a `function` can also access variables defined **outside** the function, like this:

```
var a = 4;
function myFunction() {
  return a * a;
}
```

In the last example, **a** is a **global** variable.

In a web page, global variables belong to the window object.

Global variables can be used (and changed) by all scripts in the page (and in the window).

In the first example, **a** is a **local** variable.

A local variable can only be used inside the function where it is defined. It is hidden from other functions and other scripting code.

Global and local variables with the same name are different variables. Modifying one, does not modify the other.

Variable Lifetime

Global variables live as long as your application (your window / your web page) lives.

Local variables have short lives. They are created when the function is invoked, and deleted when the function is finished.

A Counter Dilemma

Suppose you want to use a variable for counting something, and you want this counter to be available to all functions.

You could use a global variable, and a `function` to increase the counter:

```
// Initiate counter
var counter = 0;

// Function to increment counter
function add() {
  counter += 1;
}

// Call add() 3 times
add();
add();
add();

// The counter should now be 3
```

There is a problem with the solution above: Any code on the page can change the counter, without calling add().

The counter should be local to the `add()` function, to prevent other code from changing it:

```
// Initiate counter
var counter = 0;

// Function to increment counter
function add() {
  var counter = 0;
  counter += 1;
}

// Call add() 3 times
add();
add();
add();

//The counter should now be 3. But it is 0
```

It did not work because we display the global counter instead of the local counter.

We can remove the global counter and access the local counter by letting the function return it:

```
// Function to increment counter
function add() {
  var counter = 0;
  counter += 1;
  return counter;
}

// Call add() 3 times
add();
add();
add();

//The counter should now be 3. But it is 1.
```

It did not work because we reset the local counter every time we call the function.

A JavaScript inner function can solve this.

JavaScript Nested Functions

All functions have access to the global scope.

In fact, in JavaScript, all functions have access to the scope "above" them.

JavaScript supports nested functions. Nested functions have access to the scope "above" them.

In this example, the inner function `plus()` has access to the `counter` variable in the parent function:

```
function add() {
  var counter = 0;
  function plus() {counter += 1;}
```

```
  plus();
  return counter;
}
```

This could have solved the counter dilemma, if we could reach the `plus()` function from the outside.

We also need to find a way to execute `counter = 0` only once.

We need a closure.

```
var add = (function () {
  var counter = 0;
  return function () {
counter += 1; return counter}
})();

add();
add();
add();

// the counter is now 3
```

The variable `add` is assigned the return value of a self-invoking function.

The self-invoking function only runs once. It sets the counter to zero (0), and returns a function expression.

This way add becomes a function. The "wonderful" part is that it can access the counter in the parent scope.

This is called a JavaScript **closure.** It makes it possible for a function to have "**private**" variables.

The counter is protected by the scope of the anonymous function, and can only be changed using the add function.